

1/1
Regular expression [Regular exp. helps in saving time & making code & complexity.]

• import re

• method of reg ex

* Regex are Case Sensitive

1) compile = s.compile(r'*Balman! Tina Fey');

2) group() = print(group()) # returns reg ex

pipe character |

• re.compile(r'Bat(man|mobile|car|bat)')

↓
pipe

• parenthesis () defines a group in regex.

• ? → (r'Bat(w)?man);

gives out only one occurrence

• * → (r'Bat(w)*man);

gives out more than one occurrence

• + → (r'Bat(w)+man);

should have atleast one occurrence

* Python default regex are greedy, so it try matching as long as possible

- findall : find all occurrence matched in reg-ex

Regex Patterns

Regex	Meaning	Ex. Match
\d	Digit	3
\D	Non-digit	A
\s	space/tab/newline	" "
\S	non-space	"A"
\w	word char (A-Z, a-z, 0-9)	hello123
\W	Non-word char	

- Ex:

→ num = re.compile(r'\d+\$')

↳ must end with digit

→ word = re.compile(r'^.at')

↳ word should end with at [Ex: cat]

→ robotcop = re.compile(r'robotcop', re.I)

↳ ^{gn}case insensitive

Project

- Phone number & Email Add Extractor

Input Validation

- means checking and ensuring that input provided by user is correct.

• using `isalpha()`

Reading & Writing files

- using `Pathlib`, import `os`

→ `p = Path("folder/file.txt")` # creates path

- `Path.cwd()` = current dir

- `Path.home()` = home dir

- Absolute Path - gives complete address of file or folder

- Relative path - gives location relative to cwd

→ import `os` location & file name

- `os.makedirs()`

creates new dir

or

`path('').mkdir()`

- `os.path.abspath()` # returns abs path

- `path('').exists()` # returns True if path exist

• open
hellofile = open(Path.home() / 'hello.txt')
hellocontent = hellofile.read()

- for reading = 'r'
- for writing = 'w'
- for appending = 'a'

Shelve Module

- in python it is used to store python objects permanently on your desk.

```
Sh = shelve.open('mydata')  
cats = ['Zophie', 'Simon', 'Pooka']
```

```
Sh['cats'] = cats
```

→ for opening the file created
shelffile = shelve.open('mydata')

```
type(shelffile)  
shelve.DbfilenameShelf
```

Project

- create a quiz file generator

Organizing files

shutil Module (shell utilities)

Path

Ex:

```
p = path.home()
```

```
shutil.copy(p/'spam.txt', p/'some folder')
```

copying file to a folder

- `shutil.copytree(p/'spam', p/'spam_backup')`

function used to copy recursively copy an entire directory tree

Moving & Renaming file

- `shutil.move('""' bacon.txt, '""' eggs '')`

↓
which you
want to
move

↓
where you
want to move

Deleting

- Using OS

→ `os.unlink(filename)`

- Using `import Send2trash`

`Send2trash.Send2trash(filename)`

Debugging

raise

- `raise Exception('This is the exception')`

- import traceback

- `try:`

- `raise Exception('This is error message')`

- `except:`

- `errorfile = open('ErrorInfo.txt', 'w')`

- `errorfile.write(traceback.format_exc())`

- `errorfile.close()`

- `print('The traceback info was written to errorinfo.txt')`

prints, capture detail of error message into a string

- assert : check if condition true

- if false raise assertion Error

Ex: `assert list[0] < list[1]`

- Logging

- Configuring logging settings [# catches info, debug, warning etc]

`logging.basicConfig(level=logging.DEBUG,`

`format = '%(asctime)s - %(levelname)s - %(message)s')`

other logging functions / level

- INFO
- WARNING
- ERROR
- CRITICAL

→ Disabling logging

`logging.disable(logging.CRITICAL)`

→ Logging to a filename

`logging.basicConfig(filename = 'myProgramLog.txt', level = logging.DEBUG, format = '%-(asctime)s - %(levelname)s - %(message)s')`

create variables for date & time
date & time in log file

`if __name__ == '__main__':`

import logging module to log messages

`logger = logging.getLogger('myProgram')`
`logger.setLevel(logging.DEBUG)`
`handler = logging.FileHandler('myProgramLog.txt')`
`logger.addHandler(handler)`

Web Scraping

- import webbrowser

Project

- Own created, directly accessing link from cmd

→ Request modules

lets you talk to API easily

Import Requests ★

```
get () => res = requests.get("link")  
# to get and fetch data from web;
```

```
status_code = # give status of request made
```

```
# to download file
```

```
res = requests.get("link")
```

```
res.raise_for_status()
```

```
playfile = open('Roms'.txt, "wb")
```

```
for chunk in res.iter_content(10000):
```

```
    playfile.write(chunk)
```

```
playfile.close()
```

• Using BS4

to extract html code into text.

→ from BS4 import BeautifulSoup.

html = "<h1>Hello</h1> <p>World</p>"

soup = BeautifulSoup(html, 'html.parser')

print(soup.h1.text)

print(soup.p.text)

★ Project: downloading XCD project comic image

→ whiteboard

CSV & JSON

- CSV - Comma Separated Value
Also to store tabular data rows & columns.

• import CSV

open $\Rightarrow$ `exfile = open('New.csv')`
read $\Rightarrow$ `exRead = csv.reader(exfile)`

list $\Rightarrow$ `exdata = list(exRead)`

To write in CSV

write $\Rightarrow$ `outfile = open('output.csv', 'w', newline='')`
`csvwrite = csv.writer(outfile)`
 $\Rightarrow$ `csvwrite.writerow(['...'])`

TSV - tab separator value

for structure

we can write,

`csvfile = open('new.tsv', 'w')`
`csvwrite = csv.writer(csvfile, delimiter='|', lineterminator='\n')`

using dict

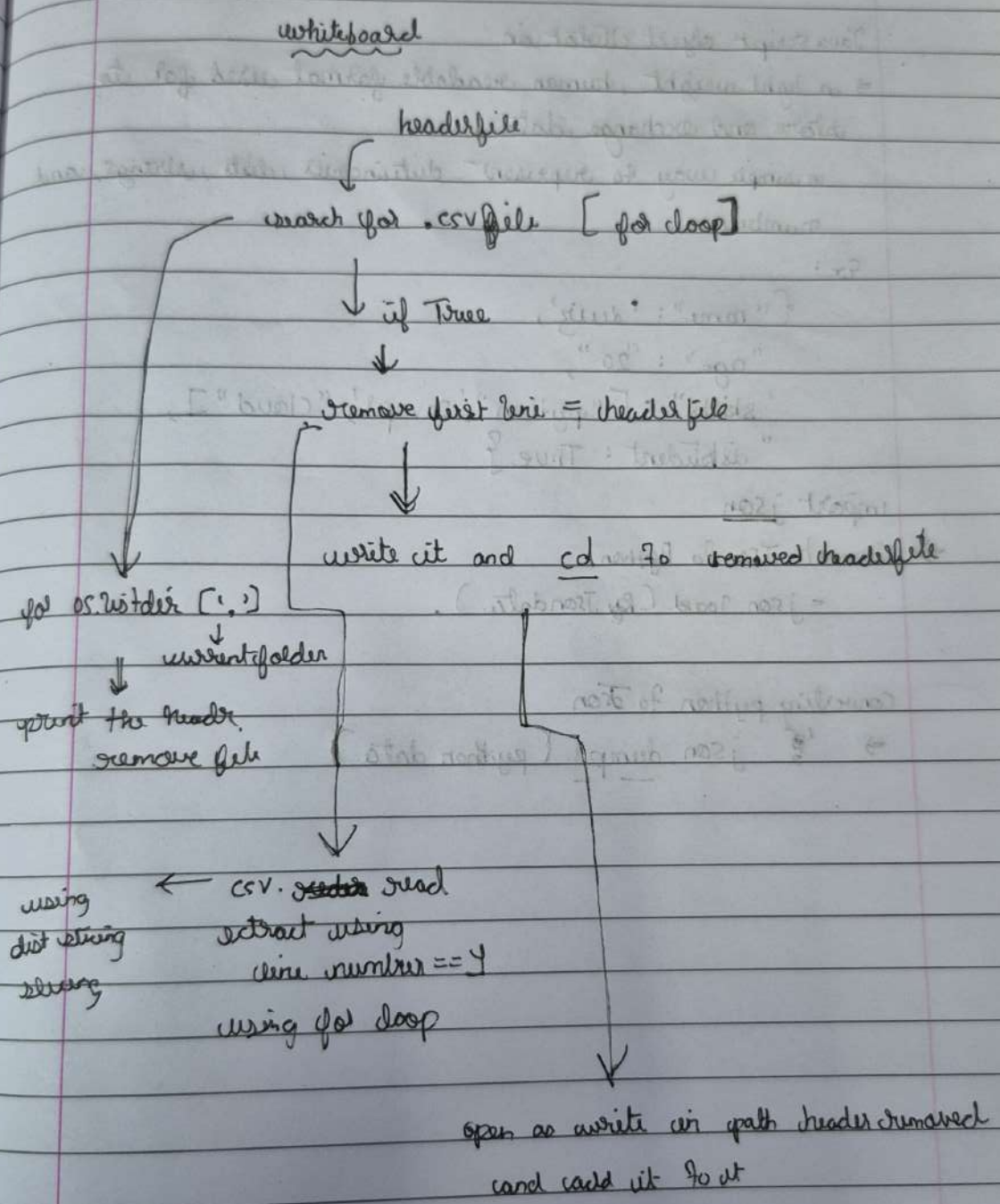
open Example in csv

$\Rightarrow$ `exdictReader = csv.DictReader(examplefile, ['time', 'name', 'amount'])`

for row in example dictreader:

`print(row['time'], row['name'], row['amount'])`

Project: Removing header files From



JSON & API

JavaScript Object Notation

⇒ a lightweight, human readable format used for to store and exchange data.

a simple way to represent dictionaries, lists, strings, numbers

Ex:

```
{ "name": "deviz",  
  "age": "20",  
  "skills": ["python", "devops", "cloud"],  
  "isStudent": True }
```

import json

⇒ Convert Json to Python

= `json.loads (Json data)`.

Converting python to Json

⇒ `json.dumps (python data)`